

Wittmann AI Interview System

Full technical and user manual for candidate interview application

Project folder: E:\wittmann_interview_ai

Generated: 16-07-2026 05:48 PM

1. Purpose and Scope

This application is the candidate-facing AI interview system. It accepts candidate resume details, validates OTP or test links, lets the candidate select an interview role, performs a mobile-friendly camera face-center check, runs aptitude and programming rounds, monitors face visibility during the interview, scores answers, and generates a PDF report with candidate details, answers, warnings, score, and the camera-check profile photo.

It is designed for both mobile and PC candidates. For mobile camera permission, candidates should open the secure public HTTPS link in Chrome, Safari, or another full browser instead of Gmail, WhatsApp, Instagram, or other in-app browsers.

2. Main Features

- Resume upload and resume parsing for name, email, phone, degree, location, and basic eligibility information.
- 10-digit Indian mobile number validation with +91 trimming support.
- OTP/test-link based candidate entry and invite-sent confirmation page with five-minute expiry messaging.
- Role selection with allowed degree and question-set restrictions.
- Separate camera-check page before the aptitude test; face must be visible and reasonably centered before starting.
- Aptitude and programming sections with randomized role-based questions from PostgreSQL question_bank.
- 30 aptitude questions and 5 programming questions where enough active questions exist.
- Copy, paste, selection, context menu, and tab/focus proctoring protections during interview.
- Mobile-tuned proctoring sensitivity so background objects do not cause false face-out alerts.
- OpenCV frame analysis for missing face, multiple significant faces, head-turn/face-center warnings, and confidence hints.
- Answer scoring using exact answer, keyword, TF-IDF semantic similarity, and communication scoring.
- PDF report generation using ReportLab, including candidate profile photo captured during camera check.
- Admin report download endpoints protected by ADMIN_REPORT_KEY.
- Render deployment configuration with Unicorn, PostgreSQL, and persistent report storage.

3. Technology Stack and Libraries

Runtime: Python Flask web application with HTML, CSS, JavaScript, PostgreSQL, OpenCV, and ReportLab.

Pinned Python Libraries from requirements.txt

- Flask==3.0.3
- opencv-python-headless==4.10.0.84
- numpy==1.26.4
- scikit-learn==1.5.1
- nltk==3.8.1
- reportlab==4.2.2
- pypdf==4.3.1
- python-dotenv==1.0.1
- Pillow==10.4.0
- cryptography==42.0.8
- pycpg[binary]==3.2.13
- unicorn==23.0.0

Imported Libraries and Modules

- from contextlib import contextmanager
- from datetime import datetime
- from datetime import datetime, timedelta
- from dotenv import load_dotenv
- from email.message import EmailMessage
- from flask import Flask, jsonify, redirect, render_template, request, send_file, session, url_for
- from pathlib import Path
- from psycpg.rows import dict_row
- from psycpg.types.json import Jsonb
- from reportlab.lib import colors
- from reportlab.lib.pagesizes import A4
- from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
- from reportlab.pdfbase import pdfmetrics
- from reportlab.pdfbase.ttfonts import TTFont
- from reportlab.platypus import Image, LongTable, Paragraph, Spacer, Table, TableStyle
- from sklearn.feature_extraction.text import TfidfVectorizer
- from sklearn.metrics.pairwise import cosine_similarity
- from typing import Dict, List
- from urllib.parse import urlsplit, urlunsplit
- from utils.ai_wrapper import ai_wrapper
- from utils.database import (
- from utils.database import QUESTION_TOPICS, ROLES, get_db, init_db
- from utils.database import get_db
- from utils.database import get_db, init_db
- from utils.database import init_db
- from utils.question_bank import ensure_question_bank_indexes
- from utils.question_bank import ensure_question_bank_indexes, import_questions_from_json
- from utils.question_bank import ensure_question_bank_indexes, select_questions_for_role
- from utils.report import generate_pdf_report
- from utils.scoring import feedback_from_score, score_answer
- from werkzeug.middleware.proxy_fix import ProxyFix
- from werkzeug.utils import secure_filename
- from xml.sax.saxutils import escape
- import base64
- import cv2
- import json
- import numpy as np
- import os
- import psycpg
- import random
- import re
- import secrets
- import smtplib
- import sqlite3
- import sys
- import tempfile
- import urllib.parse
- import urllib.request
- import uuid

- import xml.etree.ElementTree as ET
- import zipfile

Important Library Usage

- Flask: routes, sessions, request handling, template rendering, file download, JSON APIs.
- OpenCV and NumPy: camera frame decoding, Haar cascade face detection, face size checks, and proctoring analysis.
- psycopg: PostgreSQL connection, queries, JSONB storage, and shared data model with HR portal.
- scikit-learn: TF-IDF vectorization and cosine similarity for answer scoring.
- ReportLab: interview report PDF generation and candidate photo placement.
- Pillow: image handling support for report/profile-photo processing.
- python-dotenv: local .env configuration loading.
- gunicorn: production WSGI server for Render deployment.
- Werkzeug: secure filenames and proxy middleware support.

4. Folder Structure

The following tree excludes runtime-heavy folders such as .git, venv, node_modules, reports, uploads, logs, and Python cache files. Generated academic output files are summarized when a folder is very large.

```
wittmann_interview_ai/
.env
.env.example
.gitignore
AI-Based Smart Interview System Final Review PPT.pptx
AI-Based Smart Interview System Review-2 PPT.pptx
app.py
build_manual_testing_questions.mjs
DEPLOYMENT.md
Procfile
PROJECT_REPORT_OUTLINE.md
README.md
render.yaml
requirements.txt
s project
.certs/
lan-server.crt
lan-server.key
.codex/
environments/environment.toml
.vscode/
launch.json
settings.json
data/
interview_system.db
postgres_questions.sample.json
questions.json
outputs/
final_report_assets/architecture.png
final_report_assets/code_question_bank.png
final_report_assets/code_scoring.png
final_report_assets/workflow.png
final_report_render/contact_sheet.jpg
final_report_render/final_report_render.pdf
manual_testing_questions/preview.png
maximum_originality_pdf_pages/page-1.png
maximum_originality_pdf_pages/page-2.png
maximum_originality_pdf_pages/page-3.png
maximum_originality_pdf_pages/page-4.png
originality_revised_pdf_pages/page-1.png
originality_revised_pdf_pages/page-2.png
originality_revised_pdf_pages/page-3.png
originality_revised_pdf_pages/page-4.png
review2_report/converted_review1.docx
conference_paper/assets/architecture.png
conference_paper/assets/workflow.png
conference_paper/reference_preview/page_01.png
conference_paper/reference_preview/page_02.png
conference_paper/reference_preview/page_03.png
conference_paper/reference_preview/page_04.png
conference_paper/reference_preview/page_05.png
conference_paper/reference_preview/page_06.png
conference_paper/rendered/conference_paper_render.pdf
```

```

conference_paper/rendered/contact_sheet.jpg
conference_paper/rendered/ieee_contact_sheet.jpg
conference_paper/rendered/ieee_paper_render.pdf
final_report_render/pages/page_01.png
final_report_render/pages/page_02.png
final_report_render/pages/page_03.png
final_report_render/pages/page_04.png
final_report_render/pages/page_05.png
final_report_render/pages/page_06.png
final_report_render/pages/page_07.png
... 77 more files in this folder
scripts/
import_postgres_questions.py
migrate_sqlite_to_postgres.py
seed_postgres_starter_bank.py
sync_question_metadata_to_postgres.py
trust_lan_certificate.ps1
static/
robot_automation_teaser_2020_11_web.jpg
wittmann-logo-hq.png
css/style.css
js/interview.js
templates/
admin_reports.html
base.html
camera_check.html
database.html
index.html
interview.html
invite_sent.html
otp.html
result.html
roles.html
test_link.html
tools/
build_conference_paper.py
build_final_report.py
cloudflared.exe
rewrite_conference_paper_max_originality.py
rewrite_conference_paper_originality.py
utils/
ai_wrapper.py
database.py
question_bank.py
report.py
scoring.py

```

5. Source File Responsibilities

File or folder	Responsibility
app.py	Main Flask application: candidate registration, resume parsing, OTP/test-link flow, role selection, camera check, interview pages, frame analysis API, submit API, result/report download, HTTPS handling.
utils/database.py	PostgreSQL schema initialization and data access for roles, users, interviews, answers, assigned questions, test links, and report lists.
utils/question_bank.py	Question-bank table setup, active question-set parsing, role-based randomized question selection, least-used selection updates, and JSON import.
utils/scoring.py	Exact answer, keyword, semantic, communication, and feedback scoring helpers.
utils/report.py	ReportLab PDF report creation with candidate details, score table, warnings, answers, and profile photo.
utils/ai_wrapper.py	Question generation/scoring wrapper fallback used by database seeding and scoring support.
templates/	Jinja HTML pages for landing, OTP, invite confirmation, role selection, camera check, interview, result, database, and admin reports.
static/css/style.css	Responsive UI design, candidate and proctoring layout, mobile behavior, camera-check success/error styling.
static/js/interview.js	Interview browser logic: timer, camera/proctoring calls, answer capture, section transition, copy prevention, submit flow.
scripts/	Database seeding, question import, metadata sync, SQLite-to-PostgreSQL migration, LAN certificate helper.
data/questions.json	Starter/sample local JSON question data. PostgreSQL question_bank is the live source of truth.
render.yaml / Procfile	Permanent live deployment configuration for Render and Gunicorn.

6. Web Pages and API Routes

Route	Methods	Function
/	GET, POST	index
/invite-sent	GET	invite_sent
/start-test/<token>	GET	start_test
/api/parse-resume	POST	api_parse_resume
/verify-otp	GET, POST	verify_otp
/roles	GET, POST	select_role
/camera-check	GET, POST	camera_check
/interview	GET	interview
/admin/database	GET	admin_database
/admin/reports	GET	admin_reports
/api/questions	GET	api_questions
/api/analyze-frame	POST	analyze_frame
/api/submit	POST	submit_interview
/interview/result	GET	interview_result
/download-report	GET	download_report
/download-resume	GET	download_resume
/favicon.ico	GET	favicon

Candidate flow: / -> /invite-sent or /verify-otp -> /roles -> /camera-check -> /interview -> /interview/result -> /download-report.
 Test links enter through /start-test/<token>.

7. Candidate Workflow Manual

- Step 1: Candidate opens the secure interview link in a full browser. HTTPS is required for mobile camera permissions.
- Step 2: Candidate uploads resume and enters name, email, phone, and other requested details. Phone must be exactly 10 digits after +91 is removed.
- Step 3: System parses the resume, creates or finds the candidate user, generates a token, and sends a test link by email when SMTP is configured.
- Step 4: Candidate sees a separate confirmation page stating that the test link has been sent and expires in five minutes.
- Step 5: Candidate opens the test link, selects an eligible role, then moves to the camera-check page.
- Step 6: Candidate places the face inside the center mark. When the face is visible and centered enough, a green Tested OK popup is shown and Start Aptitude Test is enabled.
- Step 7: Aptitude questions are shown first. After aptitude is completed, the programming section opens separately with a next programming button/transition.
- Step 8: During the interview, copy/paste and text selection are blocked, proctoring checks run, and alerts are recorded.
- Step 9: On submit, scores are calculated, report is created, interview record is completed, and HR can review/download report and resume.

8. Camera Check and Proctoring

The camera check uses a browser camera stream and sends frames to /api/analyze-frame. Server-side OpenCV detects faces and checks whether one significant face is visible. The latest tuning is intentionally less sensitive for mobile: if the candidate face is visible and generally centered, it shows Tested OK. Full face-out or strong head turn can still trigger a warning during the interview.

- Camera precheck asks for camera only, not microphone, to avoid mobile microphone permission failures.
- The center guide is visual and tolerant; candidate does not need pixel-perfect alignment.
- Background objects should not create an error unless a second significant human face is detected.
- The final centered frame is saved as a passport/profile photo and embedded in the PDF report.

- Browser-based screenshot blocking is not technically guaranteed by web standards; the app uses deterrents such as blocking copy/select/context menu and showing privacy overlay on page blur.

9. Database Tables

- roles: role_slug, role_name, allowed degrees, allowed question sets, aptitude_minutes, programming_minutes, delete metadata.
- users: candidate identity, email, phone, resume path, location, degree, OTP metadata.
- question_patterns: older/seed metadata for role topics and question types.
- questions: older/generated question table used by legacy helpers.
- question_bank: live editable bank with role_slug, section, topic, question text, options, answer, keywords, marks, active, usage_count, question_set, soft-delete metadata.
- test_links: token, user_id, expires_at, used_at, created_at for candidate test invitations.
- interviews: interview_id, candidate, role, status, total score, report path, shortlist status/reason, timing and review metadata.
- interview_questions: snapshot of assigned questions so later question edits do not change a finished paper.
- candidate_answers: candidate answers, score, feedback, question metadata.

10. Configuration and Environment

- DATABASE_URL: PostgreSQL connection string shared with HR portal.
- SECRET_KEY: Flask session security key.
- REPORT_DIR: directory for generated reports, uploaded resumes, and profile photos.
- COMPANY_NAME: display name in pages and report context.
- SHORTLIST_MIN_SCORE: minimum score used for automatic shortlist logic.
- ADMIN_REPORT_KEY: protects admin report/database routes.
- SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASSWORD, SMTP_FROM: email/test-link sending settings.
- ENABLE_HTTPS: local HTTPS toggle. Keep off on Render because Render already provides HTTPS.

11. Running Locally

```
cd E:\wittmann_interview_ai
python -m venv venv
.\venv\Scripts\activate
pip install -r requirements.txt
python app.py
```

For LAN/mobile testing, use HTTPS or a public HTTPS tunnel. Mobile camera APIs are blocked on plain HTTP and are often blocked inside Gmail/WhatsApp/Instagram in-app browsers.

12. Permanent Deployment

- Render blueprint is defined in render.yaml.
- Build command: python -m pip install -r requirements.txt.
- Start command: python -m gunicorn app:app.
- Persistent disk is mounted at /var/data and REPORT_DIR is /var/data/reports.
- Render supplies HTTPS at the public domain; do not force app-level HTTPS on Render.
- Free Render services may sleep after inactivity, causing first request delay. Paid plan or always-on hosting removes this delay.

13. Operational Notes and Troubleshooting

- If camera permission is blocked on mobile, open the link in Chrome/Safari and allow camera permission from browser settings.
- If candidate sees stale UI, use a cache-busted URL or clear mobile browser cache; the app adds no-cache headers for dynamic pages.
- If Render logs show gunicorn command not found, confirm gunicorn is present in requirements.txt and start command uses `python -m gunicorn app:app`.
- If reports are missing after redeploy, confirm `REPORT_DIR` uses persistent storage in production.
- If HR edits questions while a candidate is already taking the test, existing candidate papers are safe because assigned questions are snapshotted in `interview_questions`.